CS7GV5 Real-Time Animation

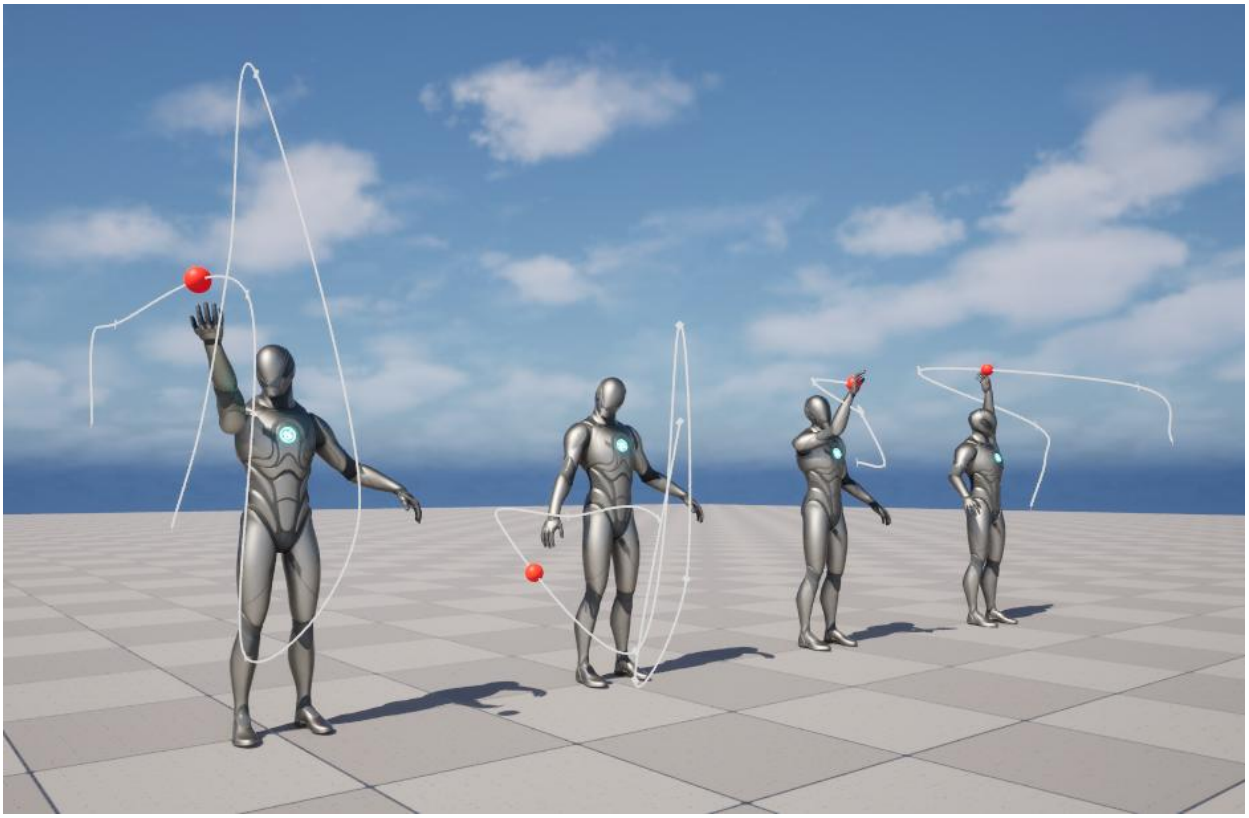
GitHub link for full code: https://github.com/Norged-Out/CS7GV5-RTA

Demo link: https://youtu.be/Q9T0WQ9PDE4

This project implements a real-time Inverse Kinematics (IK) system for a character arm in Unreal Engine using C++. The system computes joint rotations for the upper arm, lower arm, and hand so that the end effector reaches a target position. The implementation is built using Unreal Engine's Poseable Mesh component, allowing direct control over skeletal bone transforms in C++ without relying on animation blueprints.

The core IK solver is implemented using the Forward and Backward Reaching Inverse Kinematics (FABRIK) algorithm, a position-based iterative solver that operates on joint locations rather than joint angles. Compared to Jacobian or CCD approaches, FABRIK avoids expensive matrix operations and provides stable convergence suitable for real-time animation systems.

My IK chain is a three-bone arm (upper arm, forearm, and hand) with a pseudo four-bone palm-centroid targeting extension to compensate for the mannequin hand structure. Additional constraints are applied to produce natural motion, including shoulder rotation limits, elbow hinge constraints, wrist limits, and forearm roll locking. The system also includes several supporting features such as spline component used to animate the IK target and head tracking behavior.



Poseable Mesh in UE5

The provided starter project already contained a working `AAPosableCharacter` actor with a `PoseableMeshComponent` and a basic waving animation example. I used this as the base infrastructure and began my work by replacing the "waving-only" concept with a more complete idle animation mode so that the mannequin demonstrates non-trivial skeletal control even when IK is not active.

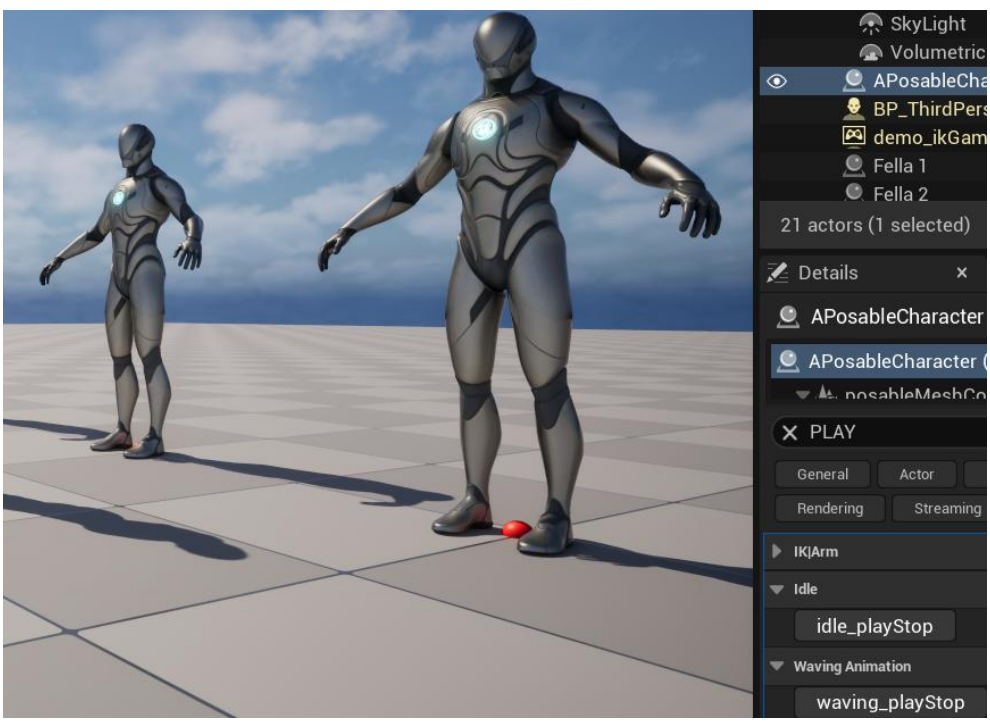
A key detail from the starter workflow is that `PoseableMesh` uses `ComponentSpace` rather than Local Space, meaning bone rotations must either be applied directly in component space or reconstructed relative to the parent transform.

Idle animation design

To avoid "drifting" or cumulative error, I capture a full snapshot of the current pose at startup: `storeCurrentPoseRotations(idle_initialBoneRotations)`. Then every frame, idle motion is generated as offsets relative to stored rest rotations: `FinalRot = BaseRot + offset`. This makes the motion stable and reversible, so the skeleton always oscillates around a valid base posture.

The idle animation introduces sinusoidal offsets to multiple bones (spine, head, clavicles, arms, pelvis, and legs). These offsets are applied relative to the stored base pose, producing subtle breathing and balance motion while the IK system is inactive.

The waving animation provided in the starter project was preserved and integrated into a enumeration-based animation state system.



Multi-bone IK in 3D

Overview of the IK Implementation

The IK solver implementation closely follows the position-based formulation described in the original FABRIK paper (Aristidou & Lasenby, 2011), where joint positions are iteratively updated through two alternating passes along the chain:

1. Backward pass: The end effector is first moved directly to the target position. Each preceding joint is then repositioned along the line toward its parent while maintaining the fixed bone length constraint.
2. Forward pass: The root joint is restored to its original position. Each subsequent joint is then repositioned along the line toward the end effector while preserving bone lengths.

These passes are repeated until the distance between the end effector and the target falls below a specified tolerance.

My IK chain consists of three joints: `upperarm_r -> lowerarm_r -> hand_r`. For each pair of adjacent joints, the bone length is computed and stored. The total arm length is also calculated, which is later used to detect unreachable targets.

```
IK_JointPositions.Empty();
IK_BoneLengths.Empty();
IK_TotalArmLength = 0.0f;
TArray<FName> Chain = {ArmRootBone,ArmMidBone,ArmHandBone};
for (FName Bone : Chain)
{
	FVector Pos = posableMeshComponent_reference
		->GetBoneLocationByName(Bone, EBoneSpaces::ComponentSpace);
	IK_JointPositions.Add(Pos);
}
for (int32 i = 0; i < IK_JointPositions.Num() - 1; ++i)
{
	float Length = FVector::Distance(IK_JointPositions[i], IK_JointPositions[i +
1]);
	IK_BoneLengths.Add(Length);
	IK_TotalArmLength += Length;
}
```

First I determine whether the target is reachable by comparing the distance between the shoulder joint and the target position to the total arm length (`DistanceToTarget <= IK_TotalArmLength`). If the target lies outside the reachable workspace, the arm is extended toward the target direction while preserving bone lengths. This behavior mirrors the unreachable target handling described in the original FABRIK formulation.

```
// Update joint positions from current mesh
for (int32 i = 0; i < NumJoints; ++i)
{
```

```
	IK_JointPositions[i] =
		posableMeshComponent_reference->GetBoneLocationByName(
			Chain[i], EBoneSpaces::ComponentSpace);
}

FVector RootPosition = IK_JointPositions[0];

// Palm centroid
FVector PalmCenter = ComputePalmCentroid();
FVector HandPos = IK_JointPositions[2];
FVector PalmOffset = PalmCenter - HandPos;

// Wrist target so palm hits sphere
FVector WristTarget = TargetPosition - PalmOffset;
float DistanceToTarget = FVector::Distance(RootPosition, WristTarget);

// Solve positions
SolveFABRIK_Positions(WristTarget, RootPosition, DistanceToTarget);

// Apply rotations
ApplyFABRIKRotations(Chain);
LockForearmRoll();
```

Iterative Position Solving

The solver performs repeated backward and forward passes over the joint chain. During the backward stage the end effector is placed at the target and joints are repositioned toward the root. During the forward stage the root is restored, and joints are propagated back toward the end effector.

The solver runs for up to 25 iterations, or until the distance between the wrist and the target falls below a tolerance threshold. This iteration count was increased from earlier experimental values to improve convergence stability during dynamic target motion.

```
for (int32 Iter = 0; Iter < IK_MaxIterations; ++Iter)
{
	// Backward pass
	IK_JointPositions.Last() = TargetPosition;
	for (int32 i = NumJoints - 2; i >= 0; --i)
	{
		FVector Dir = (IK_JointPositions[i] - IK_JointPositions[i +
1]).GetSafeNormal();
		IK_JointPositions[i] = IK_JointPositions[i + 1] + Dir *
IK_BoneLengths[i];
	}
	// Apply constraints
	ApplyShoulderConstraint();
	ApplyElbowConstraint();
	ApplyWristConstraint();
	// Forward pass
	IK_JointPositions[0] = RootPosition;
	for (int32 i = 1; i < NumJoints; ++i)
	{
		FVector Dir = (IK_JointPositions[i] - IK_JointPositions[i -
1]).GetSafeNormal();
```

```
            Dir = (Dir * (1.f - Bias) + TargetDir * Bias).GetSafeNormal();
            IK_JointPositions[i] = IK_JointPositions[i - 1] + Dir *
IK_BoneLengths[i - 1];
      }
      // Apply constraints again
      …
      if (FVector::Distance(IK_JointPositions.Last(), TargetPosition) <
IK_Tolerance) break;
}
```

Before this begins, the solver determines whether the target lies within the arm's reachable workspace. If the target distance exceeds the total arm length, the arm is extended in the direction of the target while preserving bone lengths.

```
if (DistanceToTarget > IK_TotalArmLength)
{
      FVector Dir = (TargetPosition - RootPosition).GetSafeNormal();
      for (int32 i = 1; i < NumJoints; ++i)
      {
            IK_JointPositions[i] = IK_JointPositions[i - 1] + Dir *
IK_BoneLengths[i - 1];
      }
}
```

Joint Constraints

In order to replicate real life physical limits of the joints, several constraints were integrated into the solver. Constraints are applied after both the backward and forward passes, ensuring that the corrected positions remain valid throughout the solve.

The **shoulder constraint** limits how far the upper arm can deviate from its rest direction. The current arm direction is compared against the original direction recorded during initialization. If the rotation exceeds a specified maximum angle, the vector is rotated back toward the allowed cone. This prevents the arm from rotating into physically implausible configurations.

```
float Dot = FVector::DotProduct(RestDir, LocalDir); // gets clamped
float Angle = FMath::RadiansToDegrees(FMath::Acos(Dot));
if (Angle > MaxShoulderAngle)
{
      float Excess = Angle - MaxAngle;
      FVector Axis = FVector::CrossProduct(RestDir, LocalDir).GetSafeNormal();
      FQuat Correction = FQuat(Axis, FMath::DegreesToRadians(-Excess));
      LocalDir = Correction.RotateVector(LocalDir);
}
FVector NewDir = GetActorTransform().TransformVectorNoScale(LocalDir);
NewDir.Normalize();
// Rebuild elbow position
IK_JointPositions[1] = Shoulder + NewDir * IK_BoneLengths[0];
```

The **elbow constraint** projects the forearm onto a stable hinge plane defined relative to the torso direction. A pole-vector approach is used to maintain consistent bending direction. The angle

between the upper arm and forearm is then clamped between minimum and maximum bend limits. This approach replaced earlier plane-based constraints that proved unstable.

```
// Compute a stable pole vector
FVector StablePole = FVector::CrossProduct(BodyForward,
ShoulderToHand).GetSafeNormal();
// Plane normal defining elbow hinge plane
FVector PlaneNormal = FVector::CrossProduct(ShoulderToHand,
StablePole).GetSafeNormal();
if (!PlaneNormal.IsNearlyZero())
{
	// Project elbow onto the hinge plane
	FVector ShoulderToElbow = Elbow - Shoulder;
	FVector Projected =	FVector::VectorPlaneProject(ShoulderToElbow,
PlaneNormal).GetSafeNormal();
	IK_JointPositions[1] =
		Shoulder + Projected * IK_BoneLengths[0];
}
// Clamp elbow bend angle
…
// Enforce min/max elbow bend
if (Angle < IK_MinElbowAngle || Angle > IK_MaxElbowAngle)
{
	float Clamped = FMath::Clamp(Angle, IK_MinElbowAngle, IK_MaxElbowAngle);
	FVector Axis = FVector::CrossProduct(Upper, Lower).GetSafeNormal();
	FQuat Correction = FQuat(Axis, FMath::DegreesToRadians(Delta));
	Lower = Correction.RotateVector(Lower);
	// Rebuild wrist position with corrected forearm direction
	IK_JointPositions[2] = IK_JointPositions[1] + Lower * IK_BoneLengths[1];
}
```

The **wrist constraint** is applied using a cone-based limit relative to the forearm direction. If the wrist bends beyond the allowed angle, the direction vector is rotated back toward the allowable region. This prevents excessive bending and reduces unnatural poses near extreme targets.

```
if (Angle > IK_WristMaxAngle)
{
	float Excess = Angle - MaxAngle;
	FVector Axis = FVector::CrossProduct(ParentDir, WristDir).GetSafeNormal();
	FQuat Correction = FQuat(Axis, FMath::DegreesToRadians(-Excess));
	WristDir = Correction.RotateVector(WristDir);
}
// Rebuild wrist position using corrected direction
IK_JointPositions[2] = Elbow + WristDir * IK_BoneLengths[1];
```

Rotation Reconstruction

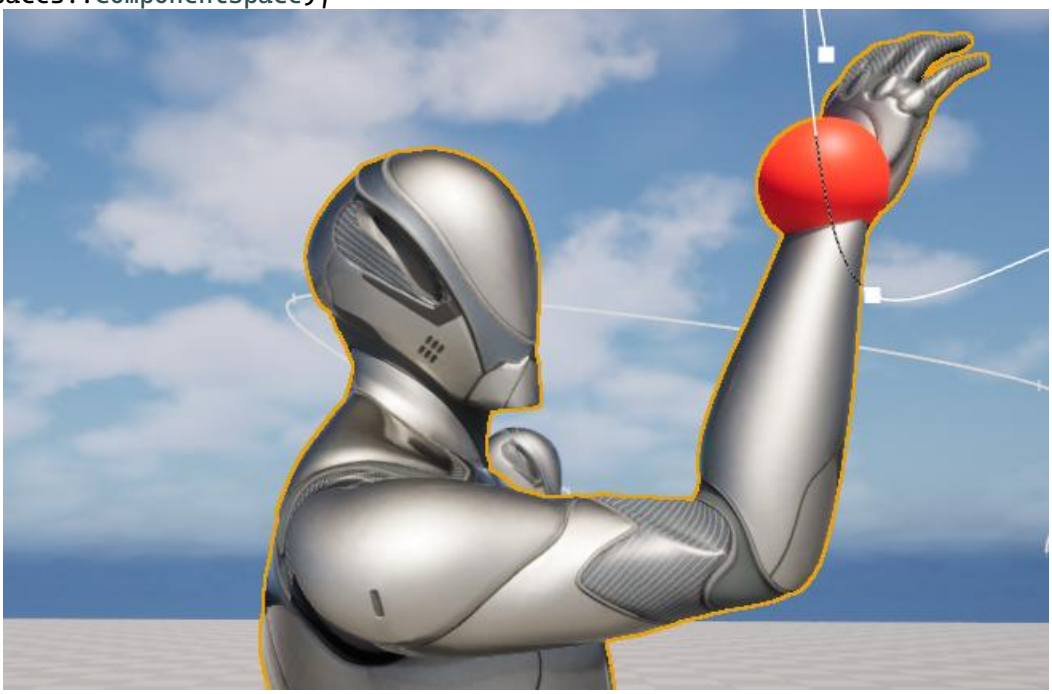
After joint positions have been solved, I convert these positions into bone rotations. For each bone, the current direction from the mesh is compared to the desired direction obtained from the solved joint positions. A quaternion rotation that aligns these vectors is then applied to the bone. This step translates the position-based FABRIK solution into skeletal rotations that deform the mesh correctly.

```
FQuat Delta = FQuat::FindBetweenNormals(CurrentDir, DesiredDir);

FTransform CurTransform
      = posableMeshComponent_reference->GetBoneTransformByName(Bone,
EBoneSpaces::ComponentSpace);

FQuat NewRot = Delta * CurTransform.GetRotation();

posableMeshComponent_reference->SetBoneRotationByName(Bone,NewRot.Rotator(),
EBoneSpaces::ComponentSpace);
```



Palm Centroid Targeting

During early testing, I observed that the Unreal mannequin's hand bone behaves more like a wrist joint, with the actual palm area defined by several metacarpal bones. As a result, targeting the hand bone directly caused the solver to reach only the wrist rather than the palm.

To solve this issue, I compute the centroid of four metacarpal bones: `index_metacarpal_r`, `middle_metacarpal_r`, `ring_metacarpal_r`, `pinky_metacarpal_r`. The average position of these bones is treated as an approximate palm location, and the wrist target is offset so the palm centroid aligns with the sphere. This produces a more natural reaching behavior and effectively acts as a pseudo four-bone extension of the chain.

```
for (FName Bone : PalmBones)
{
      Sum += posableMeshComponent_reference->GetBoneLocationByName(
      Bone,EBoneSpaces::ComponentSpace);
      Count++;
}

return Sum / Count;
```

Scripted Animation for the End Effector

To demonstrate the IK solver under dynamic conditions, the target sphere is animated along a spline path. The class includes a `SplineComponent` attached to the character actor. The target position is sampled along the spline based on elapsed time.

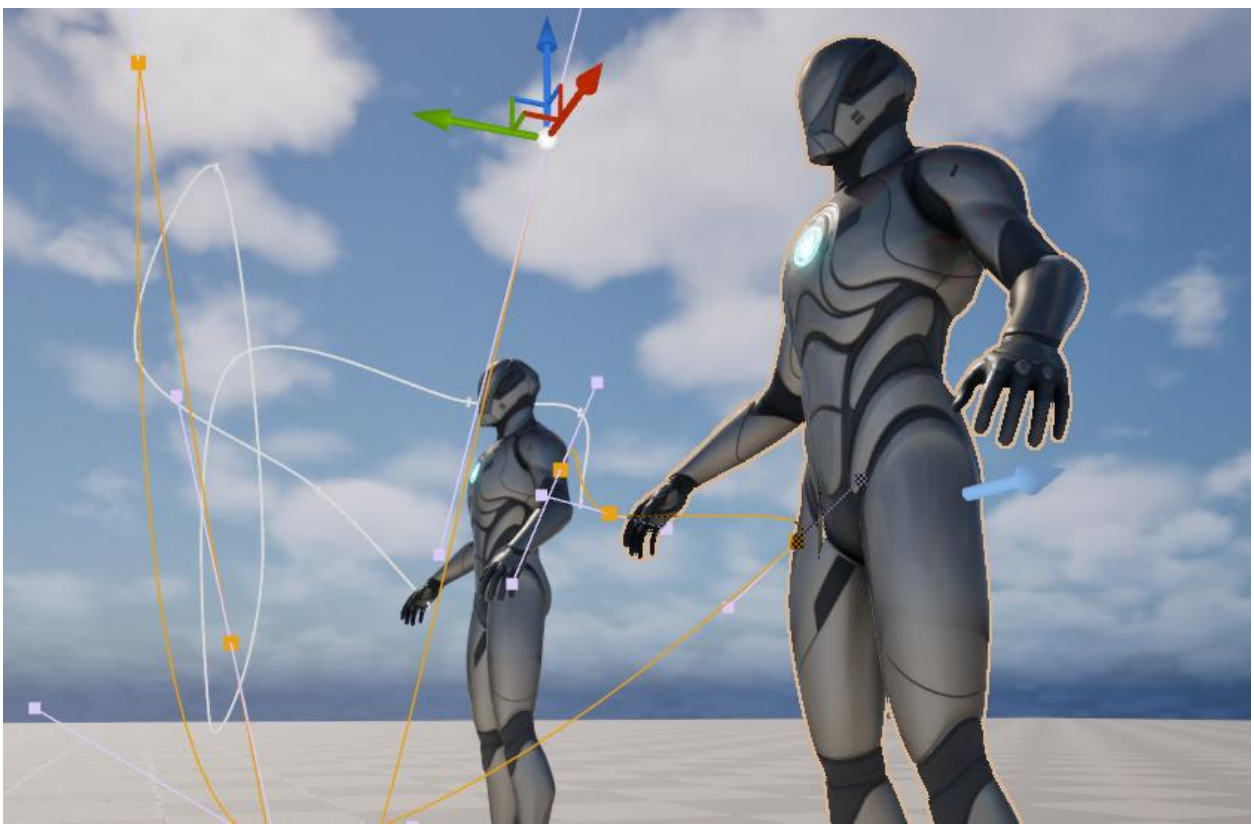
```
// normalize 0->1 over duration
float Alpha = FMath::Fmod(SplineTime / SplineDuration, 1.f);

// ease in/out motion
Alpha = FMath::InterpEaseInOut(0.f, 1.f, Alpha, 2.f);

// convert alpha to spline distance
float Distance = Alpha * HandPathSpline->GetSplineLength();

// sample spline position
FVector SplinePos = HandPathSpline->GetLocationAtDistanceAlongSpline(
            Distance,ESplineCoordinateSpace::World);
```

As the sphere moves along the spline, the IK solver updates each frame, so the arm continuously tracks the moving target and produces smooth motion.



Aside from this, the character still retains the two procedural animation modes (idle and wave).

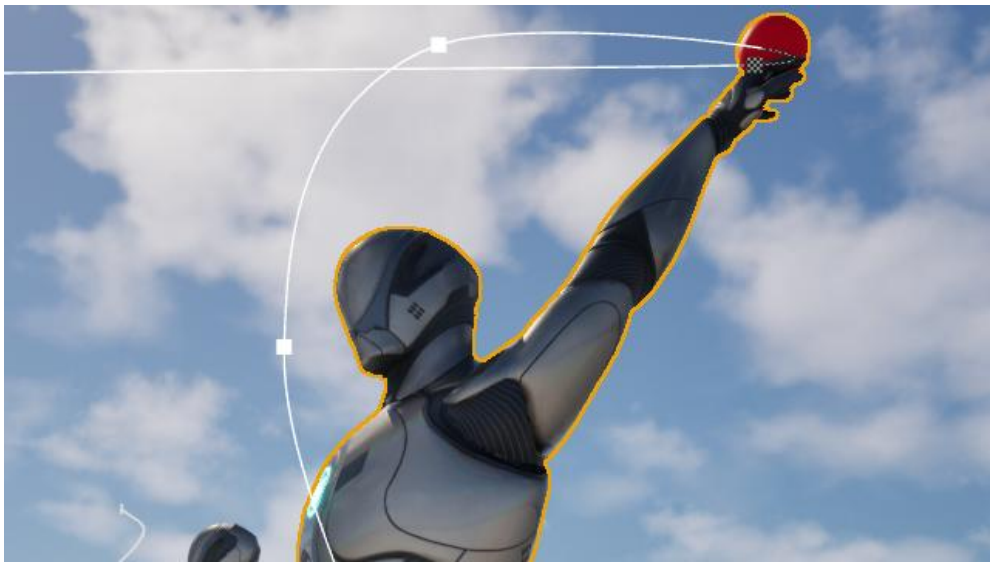
Advanced Features

1. Palm-Centroid Targeting

The palm centroid system improves reaching accuracy and compensates for limitations in the mannequin hand structure.

2. Flexible Bone Mapping

The IK chain bone names are exposed as editable properties. This allows the same solver to operate on different limbs or mirrored characters simply by modifying the bone names. This was used to demonstrate both right arm and left arm IKs on the project.



3. Head Look-At System

When the IK arm mode is active, the character's head also rotates to track the target sphere. The rotation is computed from the direction vector between the head and the target and distributed between the neck and head bones. This creates the impression that the character visually follows the object it is reaching for.

```
FRotator LookRot = Dir.Rotation();

FRotator TargetRot = UKismetMathLibrary::NormalizedDeltaRotator(LookRot,
GetActorRotation());
CurrentHeadRot = FMath::RInterpTo(CurrentHeadRot, TargetRot, GetWorld()-
>GetDeltaSeconds(), 5.f);
FRotator LocalRot = CurrentHeadRot;

// clamp natural limits
LocalRot.Yaw = FMath::Clamp(LocalRot.Yaw, -HeadYawLimit, HeadYawLimit);
LocalRot.Pitch = FMath::Clamp(LocalRot.Pitch, -HeadPitchLimit, HeadPitchLimit);
LocalRot.Roll = 0.f;

// split motion across neck/head
FRotator NeckOffset = LocalRot * 0.35f;
FRotator HeadOffset = LocalRot * 0.45f;

// apply rotations relative to rest pose
…
```

4. Forearm Roll Stabilization

A roll-locking mechanism ensures that the forearm inherits roll rotation from the upper arm. This reduces twisting artifacts that can occur when the solver aligns bones purely based on direction vectors.

```
LowerRot.Roll = UpperRot.Roll;
posableMeshComponent_reference->SetBoneRotationByName(Lower,LowerRot,
      EBoneSpaces::ComponentSpace);
```

Additional Experimental Features

Several additional features were explored but ultimately excluded due to stability or complexity issues. These included extending the solver to support leg IK, which proved unstable due to different constraint requirements such as knee direction and foot orientation. Automatic finger curling near the target sphere was also tested but produced unnatural deformation and would require controlling full finger chains (fifteen additional bones). A clavicle assist system was briefly implemented using a small cone constraint but caused occasional mesh intersections due to solver interactions. These experiments did help improve the stability of the final IK system.